



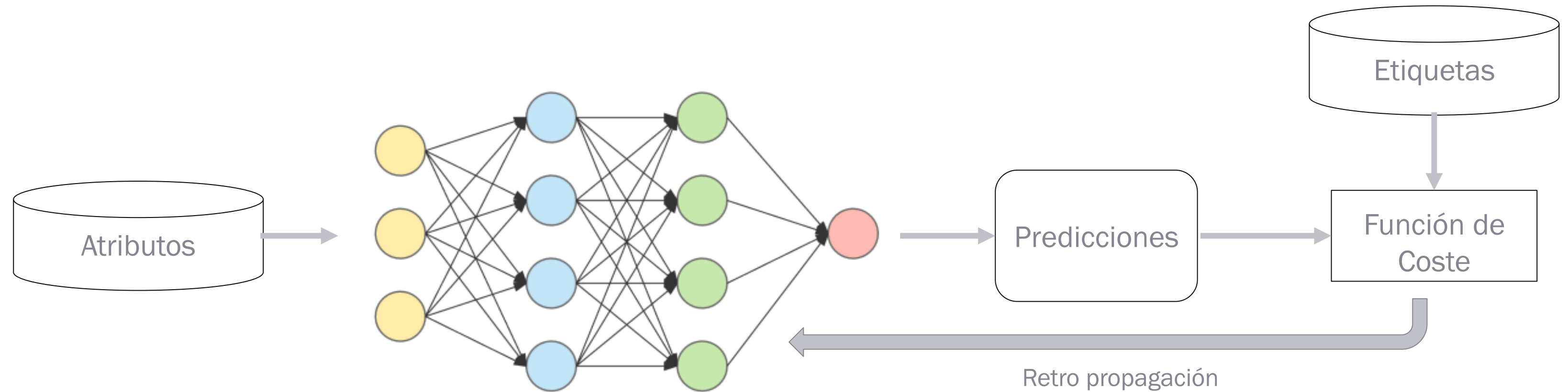
Aspectos prácticos del Entrenamiento (II)

Redes Neuronales y Aprendizaje Profundo

01

Ciclo de Entrenamiento

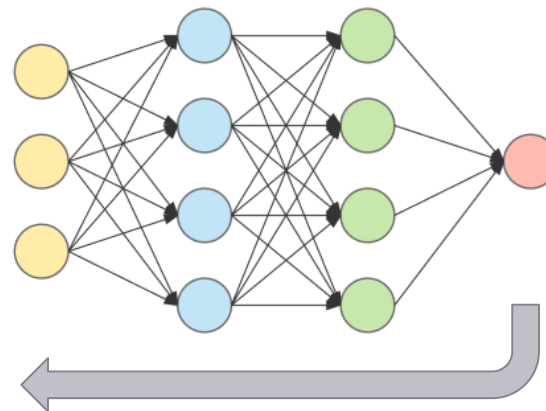
Ciclo de Entrenamiento



¿Cuándo se actualizan los pesos?



Full Batch Gradient Descent



Se corrigen los pesos tras evaluar todos las instancias.

PROs

- Más preciso
- Paralelizable

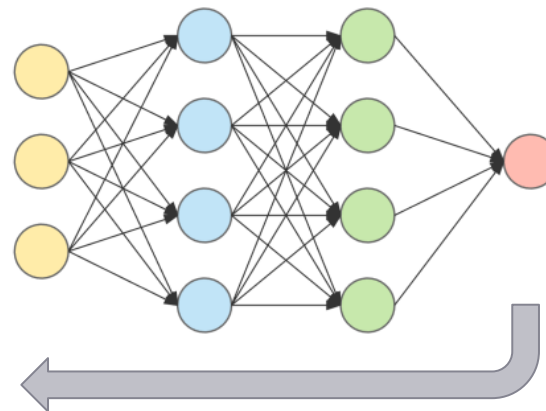
CONs

- Necesidad de memoria
- Lento

Online



...



Se corrigen tras pasar cada instancia

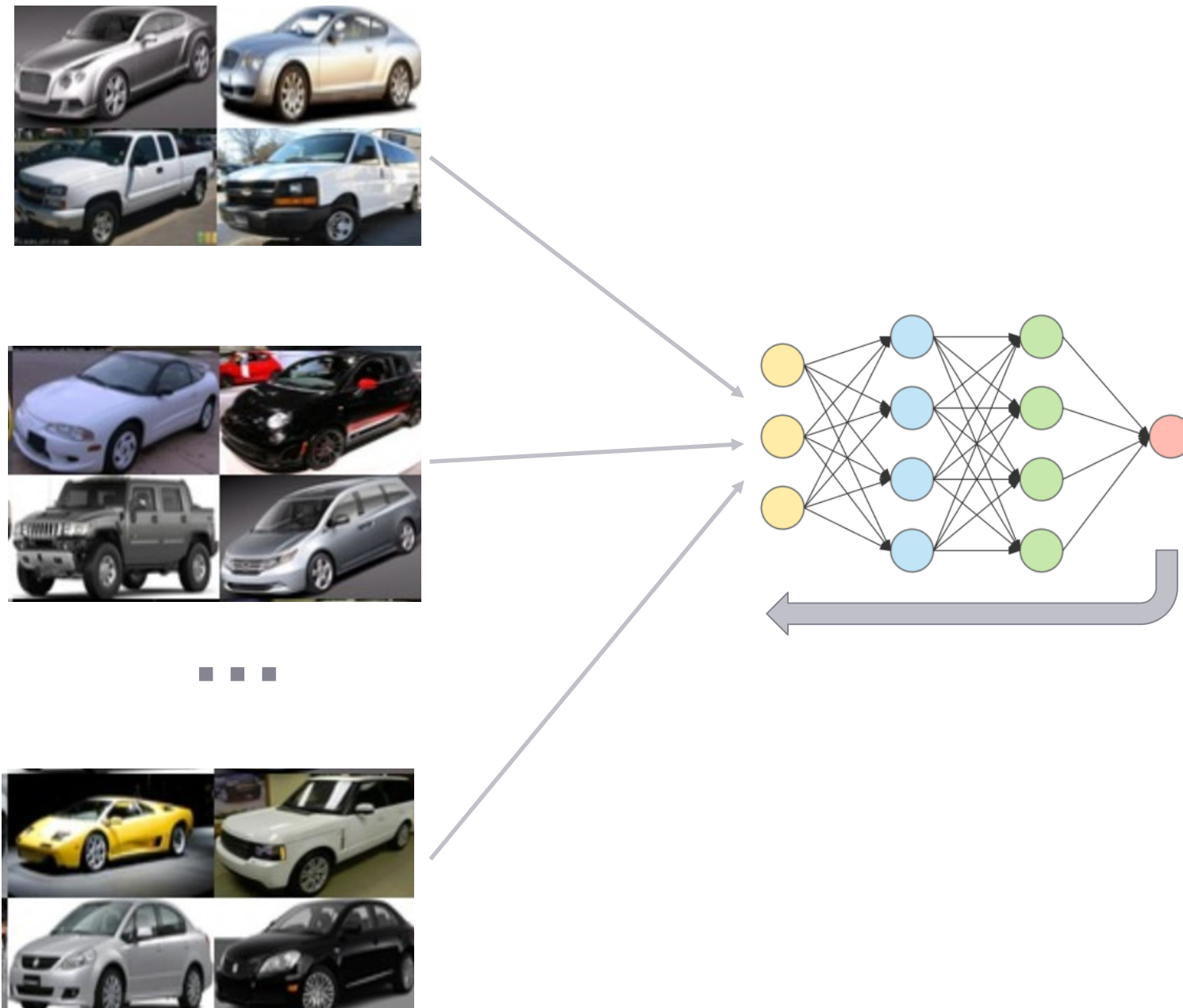
PROs

- Más rápido
- Adaptable a cambios

CONs

- Se olvida de las instancias

Mini-Batch Gradient Descent



- Se generan lotes balanceados
- Requiere menos cálculos
- Paralelizable

Otros aspectos



Iteracción: una instancia



Batch-size: un grupo de instancias



Epoch: todas las instancias

02

Optimización Avanzada

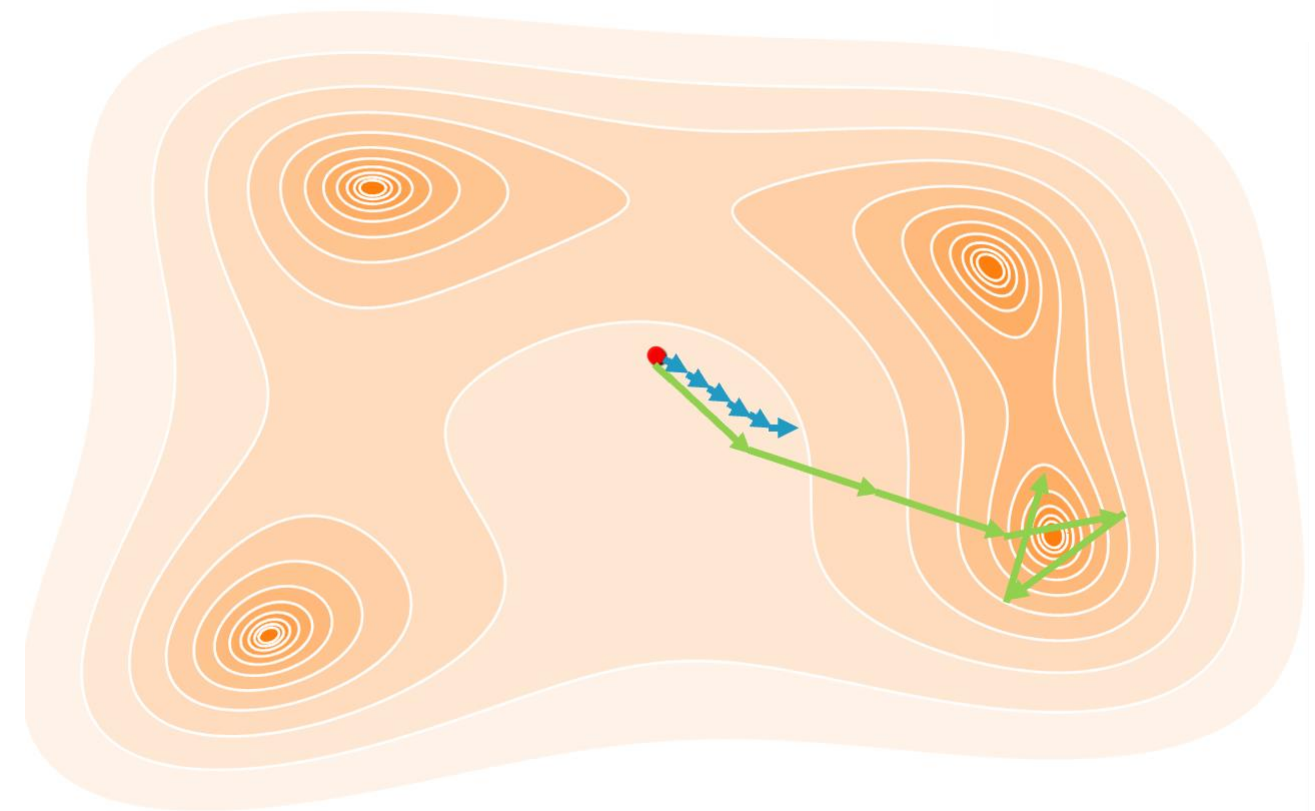
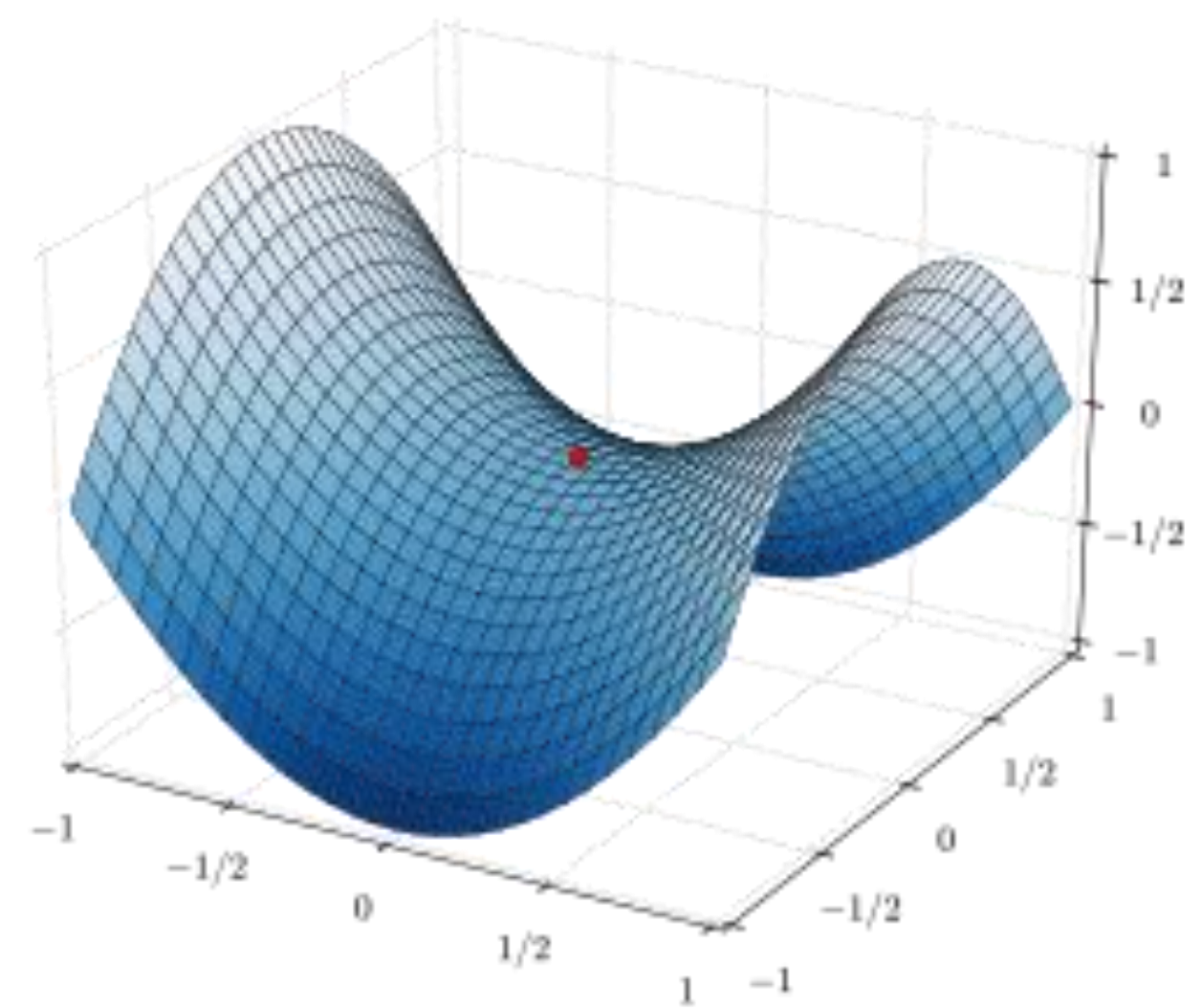
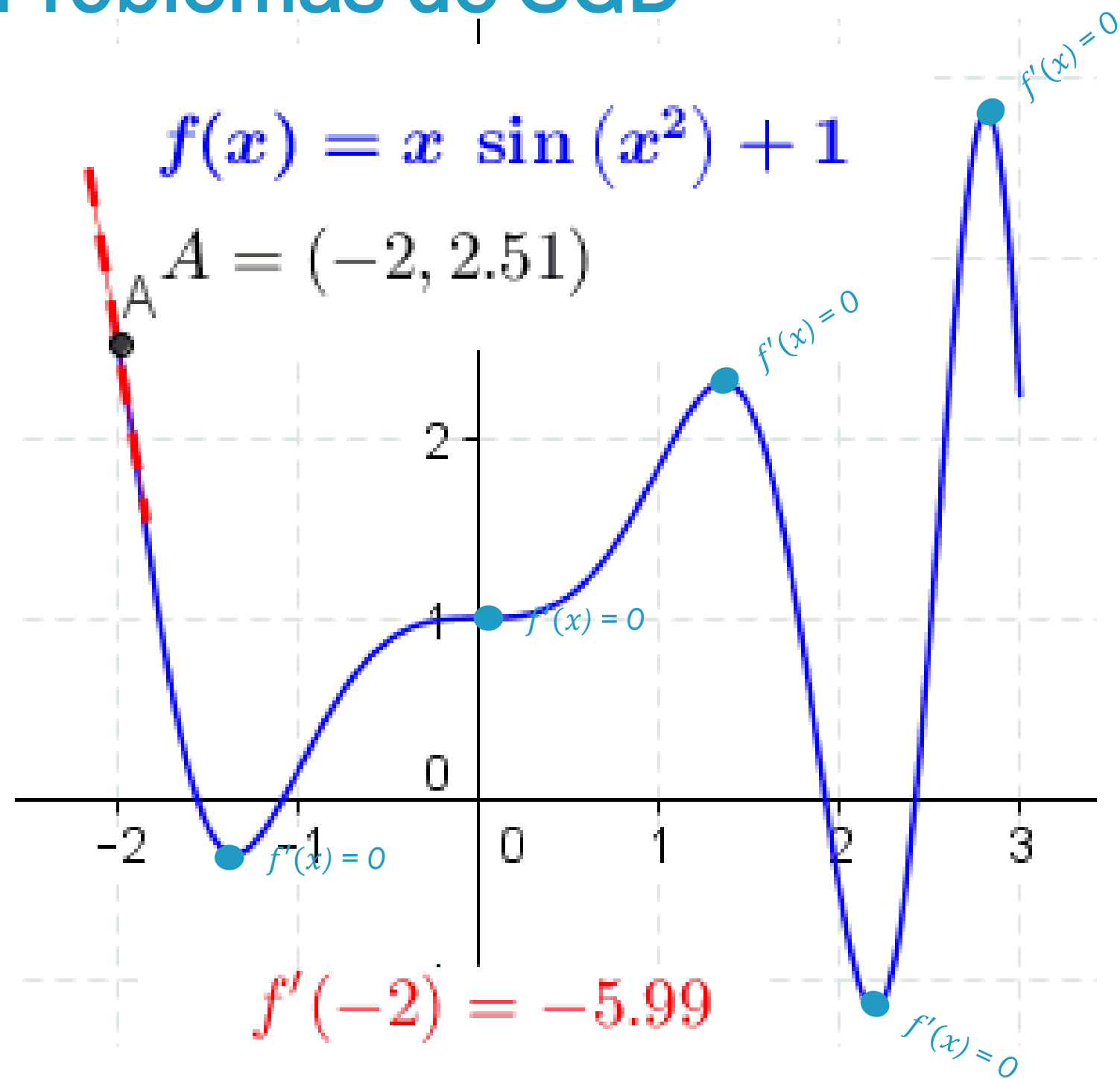
Recordemos lo que hace un optimizador:

- Son algoritmos cuya función es ajustar los pesos del modelo durante el entrenamiento para minimizar la función de pérdida.
- El optimizador utiliza el gradiente de la función de pérdida (calculado mediante el backpropagation), para actualizar esos pesos y disminuir el error.

Optimizadores más comunes:

- **Stochastic Gradient Descent (SGD):** Actualiza los pesos en cada mini-batch.
- **Momentum:** Añade una “memoria” de las actualizaciones anteriores para acelerar la convergencia.
- **RMSProp:** Ajusta el learning rate individualmente para cada parámetro usando la media cuadrática de los gradientes recientes.
- **Adam:** Combina momentum y RMSProp. Es el más utilizado.

Problemas de SGD



Problemas de SGD

- Convergencia lenta si la superficie de la función de pérdida es muy compleja.
- Trayectoria inestable, lo cual puede llevar a que el “loss” fluctue mucho.
- Estancamiento en mínimos locales o puntos de silla (SGD no los distingue), el modelo deja de aprender incluso si no ha alcanzado una solución óptima.

SGD con momentum



$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$



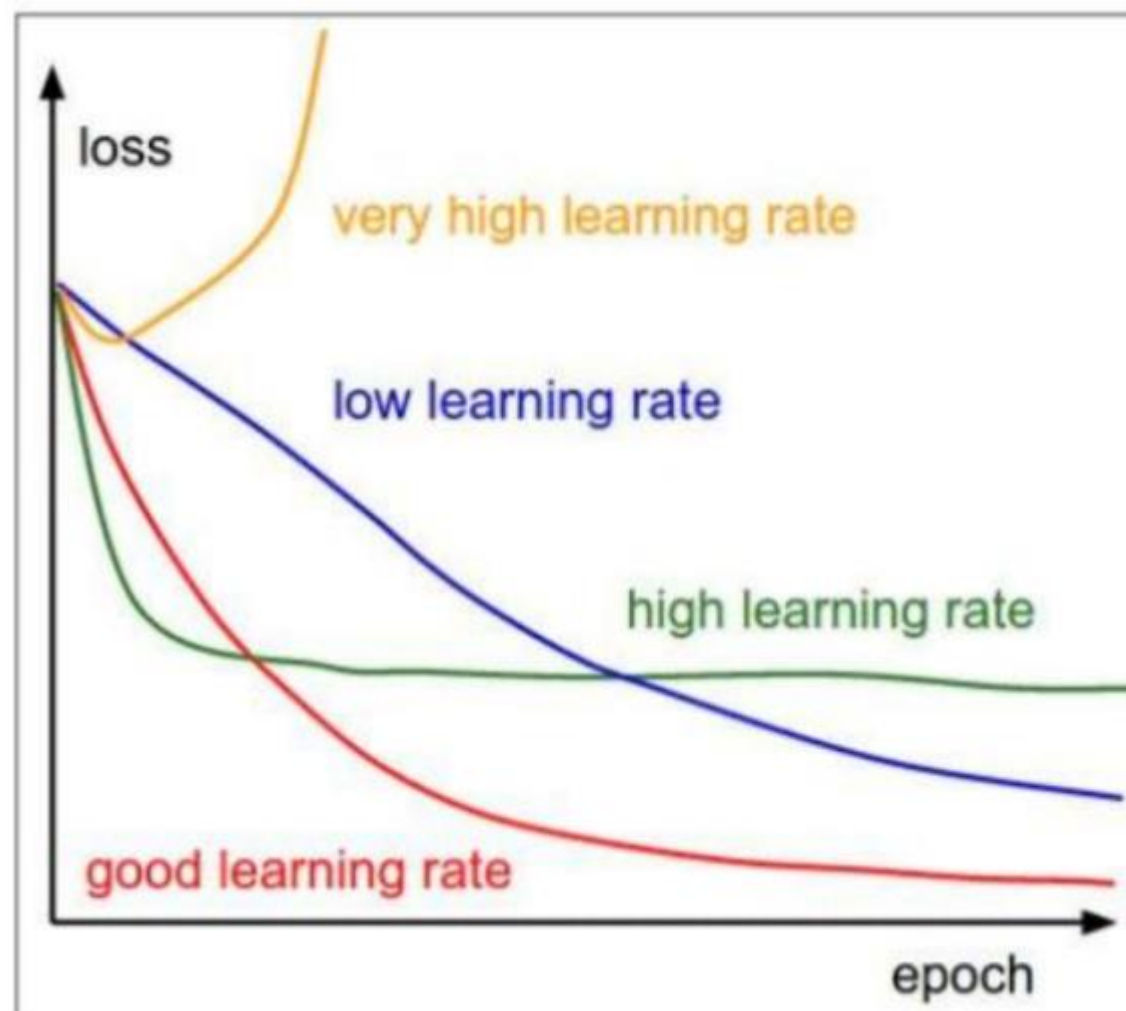
SGD con momentum

- El “momentum” ayuda a acelerar en la dirección adecuada, acumulando velocidad.
- Reduce las oscilaciones, especialmente en direcciones con gradientes variables.



- Sin “momentum”: Se detiene o rebote en las paredes del valle.
- Con “momentum”: Acumula velocidad y sigue adelante, ignorando pequeñas subidas y bajadas.

Learning rate adaptativos



AdaGrad

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared += dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

RMSProp

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Adam

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

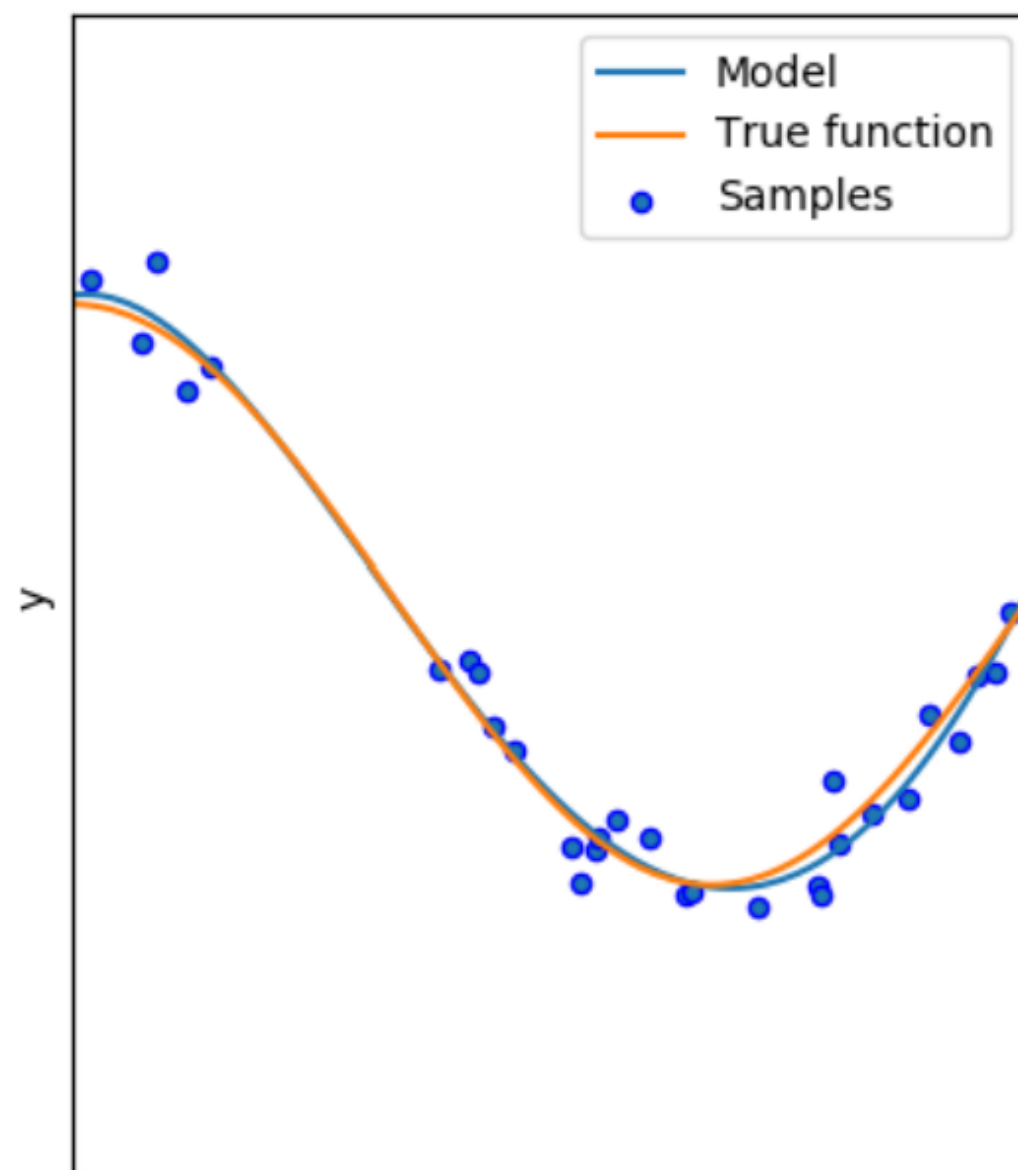
AdaGrad / RMSProp

03

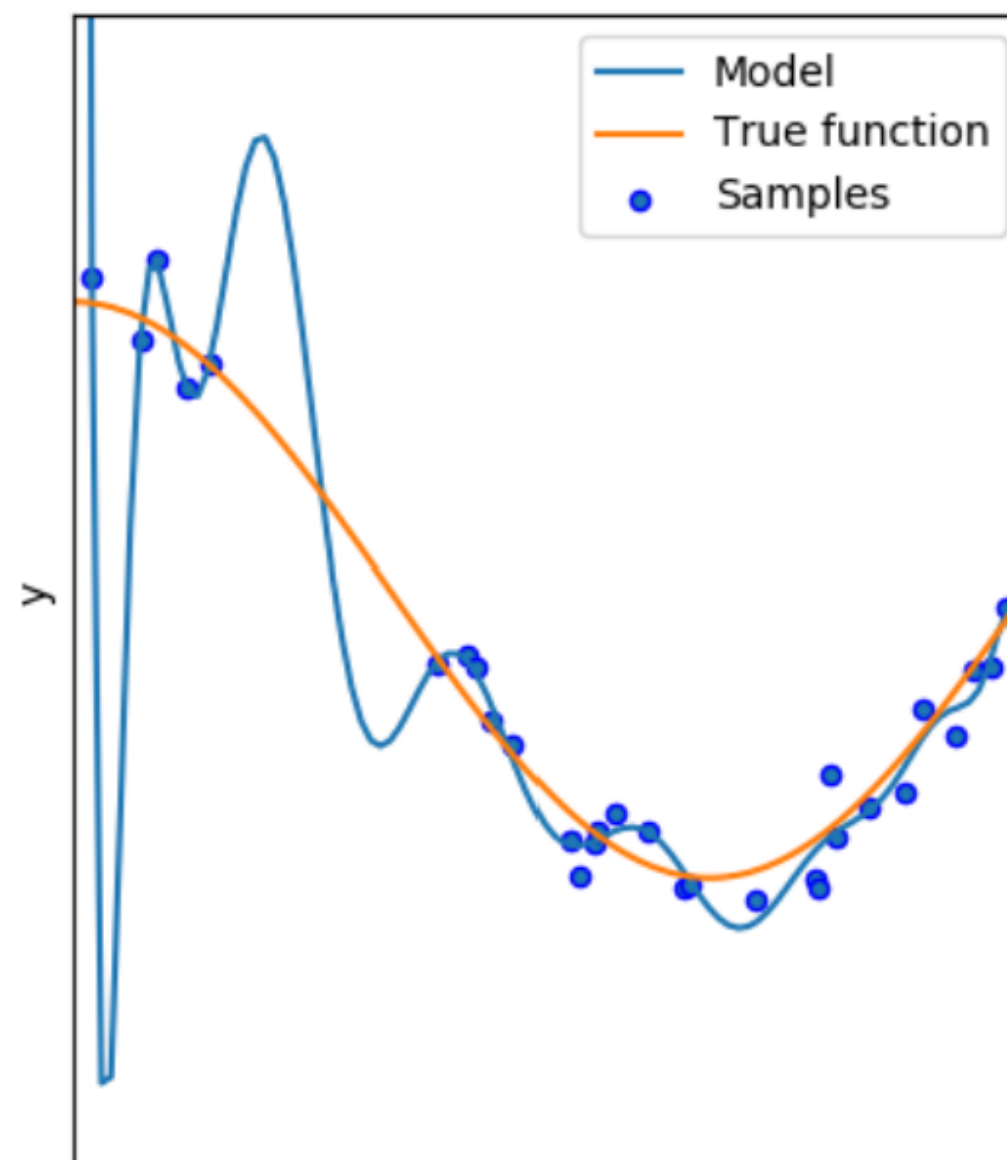
Regularización

Overfitting

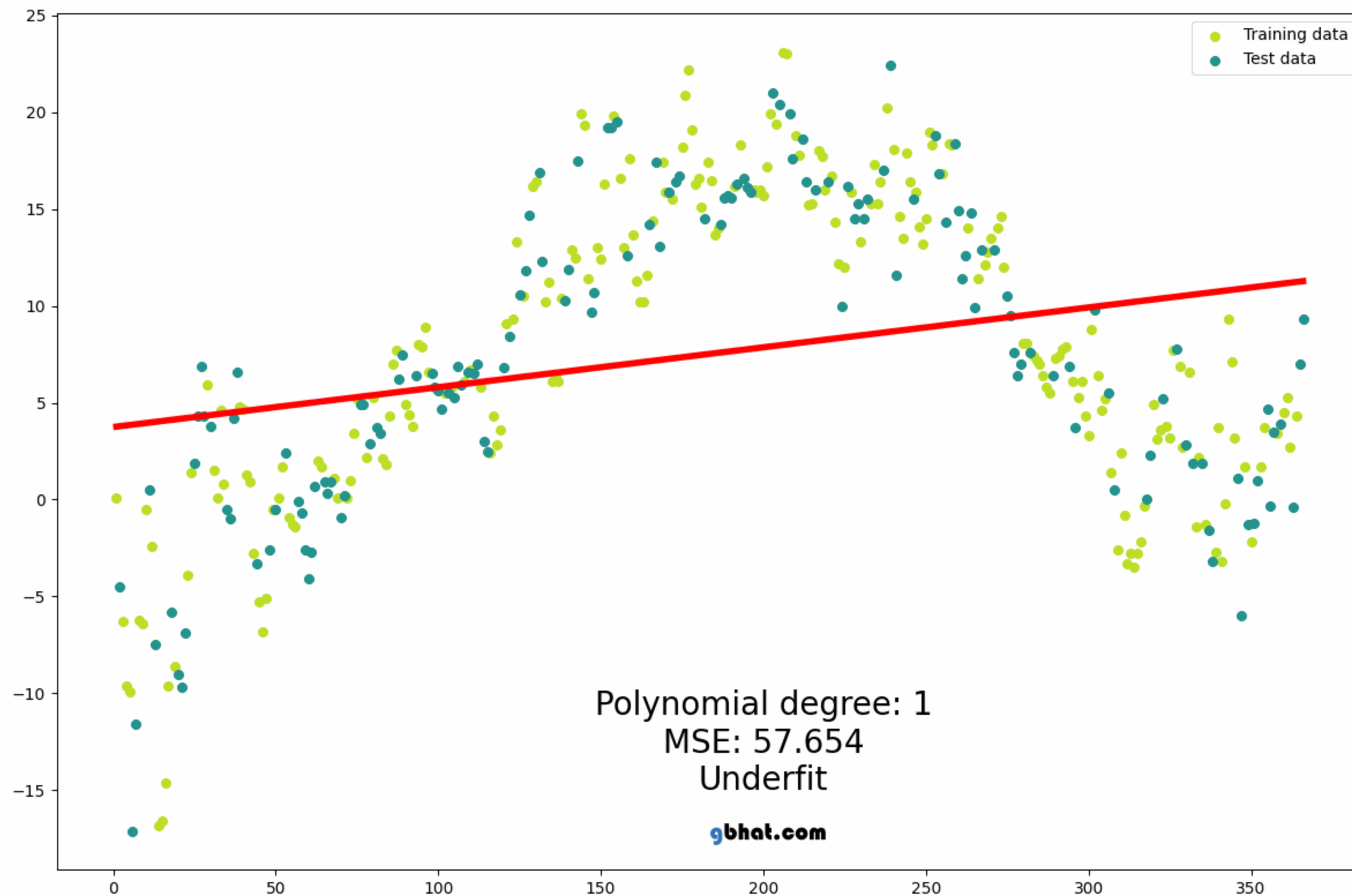
Degree 4



Degree 15

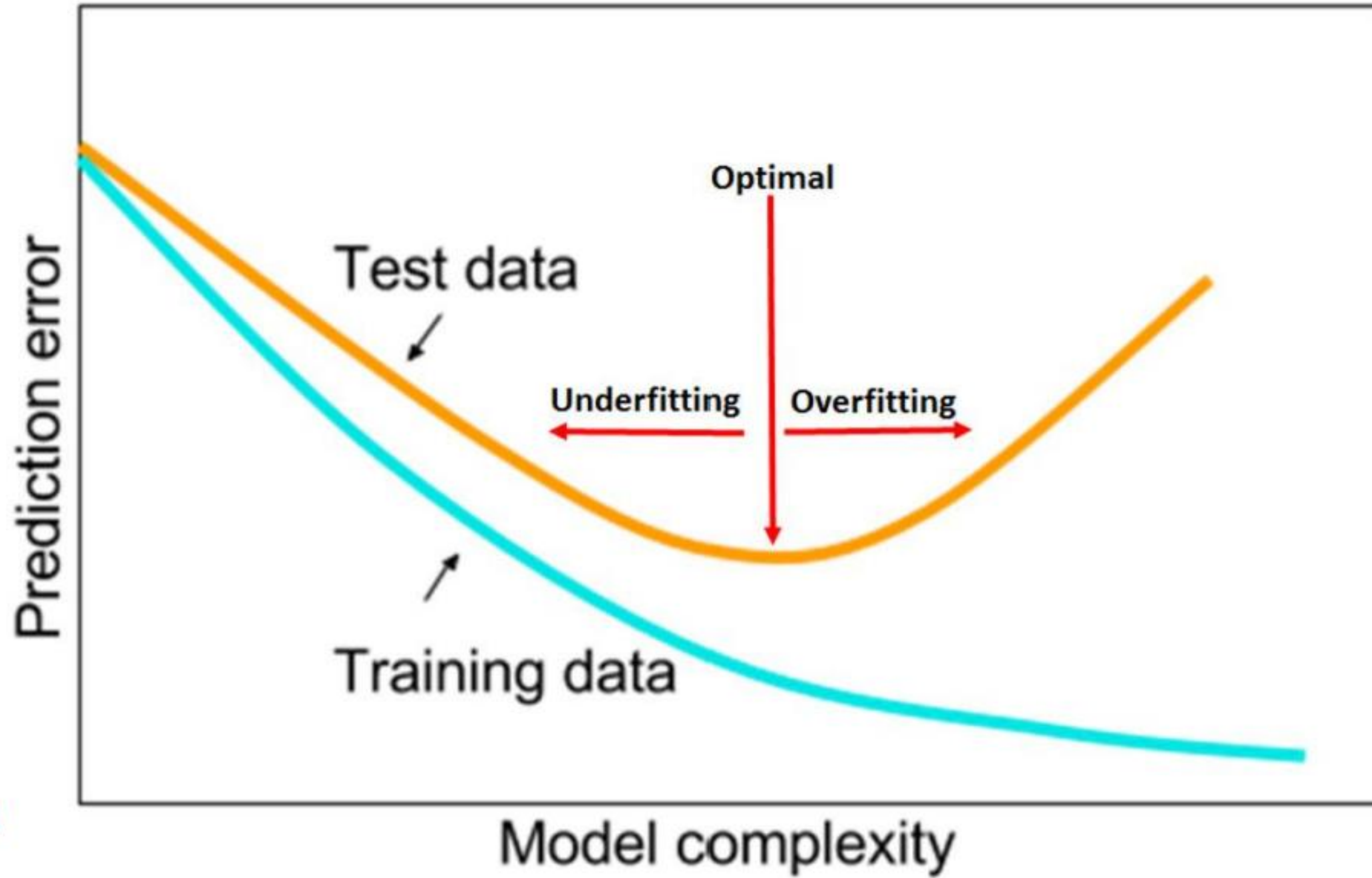


Overfitting



[Fuente](#)

Overfitting



Regularización L1 y L2

L1 (*Lasso*)

Función de coste
original

$$C = C_0 + \frac{\lambda}{n} \sum_w |w|$$

Penalización

Favorece la “sparsity” (algunos pesos se vuelven 0)

L2 (*Ridge*)

Función de coste

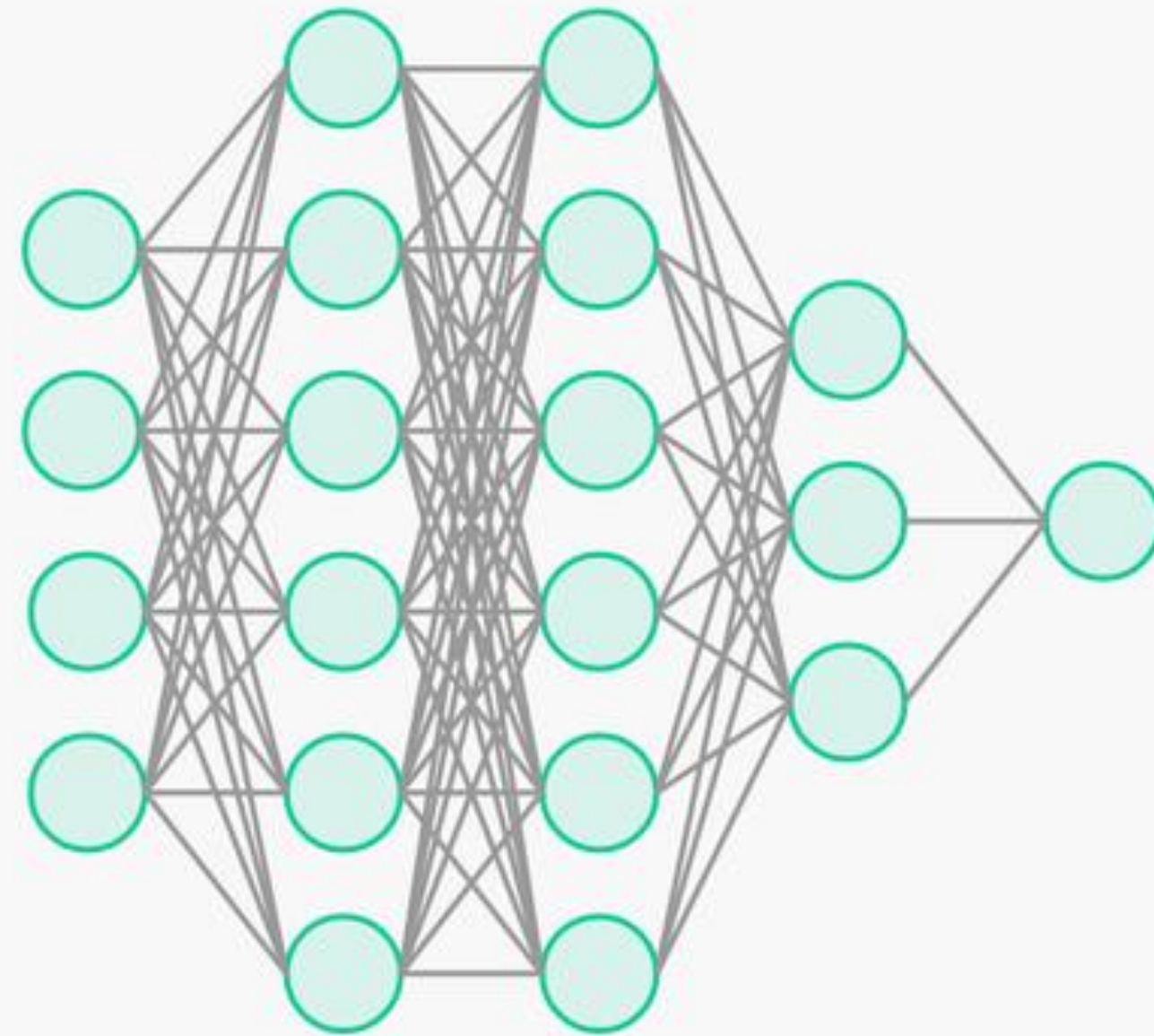
$$C = -\frac{1}{n} \sum_{x_j} \left[y_j \ln a_j^L + (1 - y_j) \ln(1 - a_j^L) \right] + \frac{\lambda}{2n} \sum_w w^2$$

Penalización

Favorece pesos pequeños, sin eliminarlos

ElasticNet : combinación de L1 y L2

Dropout



$$p^{[0]} = 0.0$$

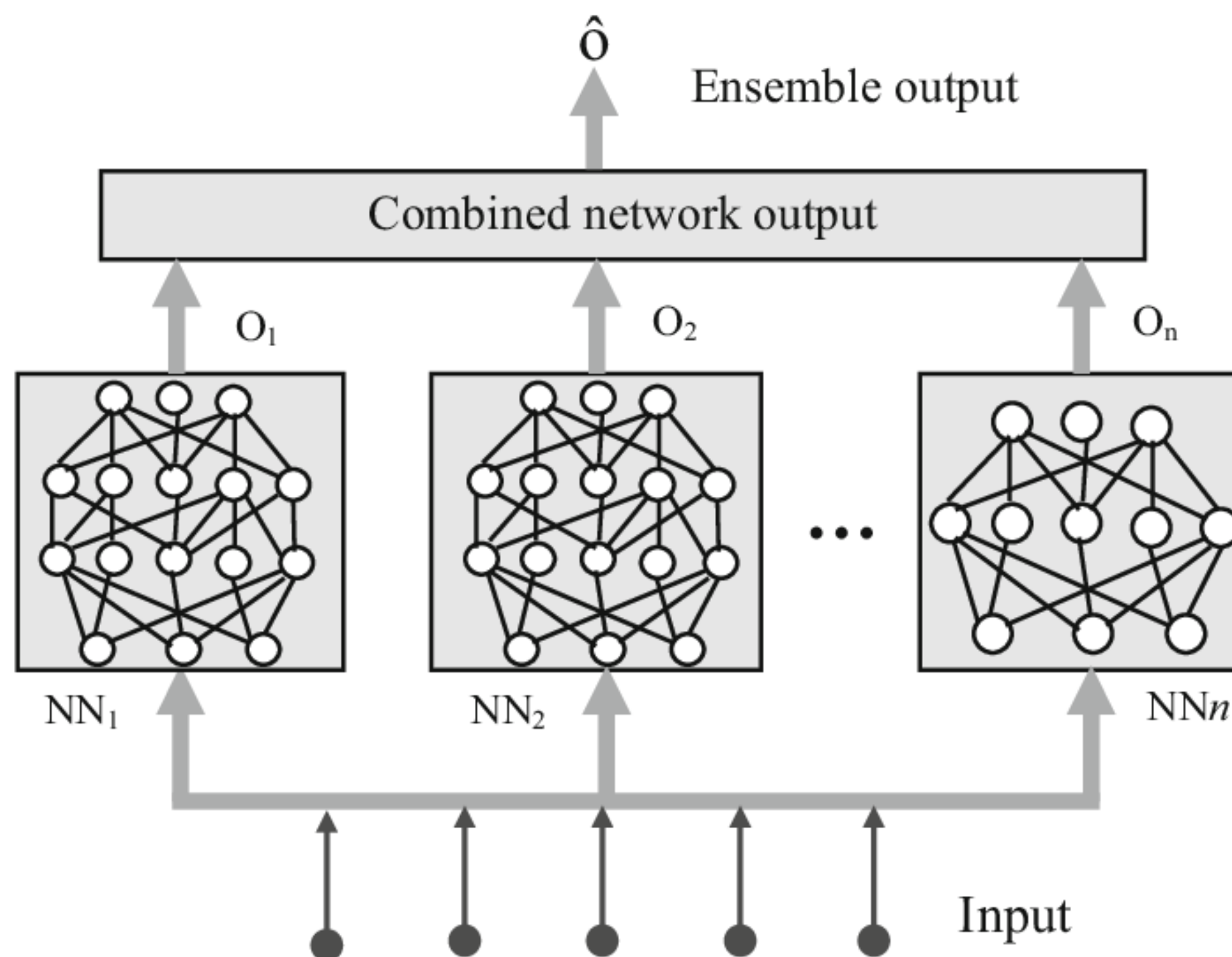
$$p^{[1]} = 0.5$$

$$p^{[2]} = 0.33$$

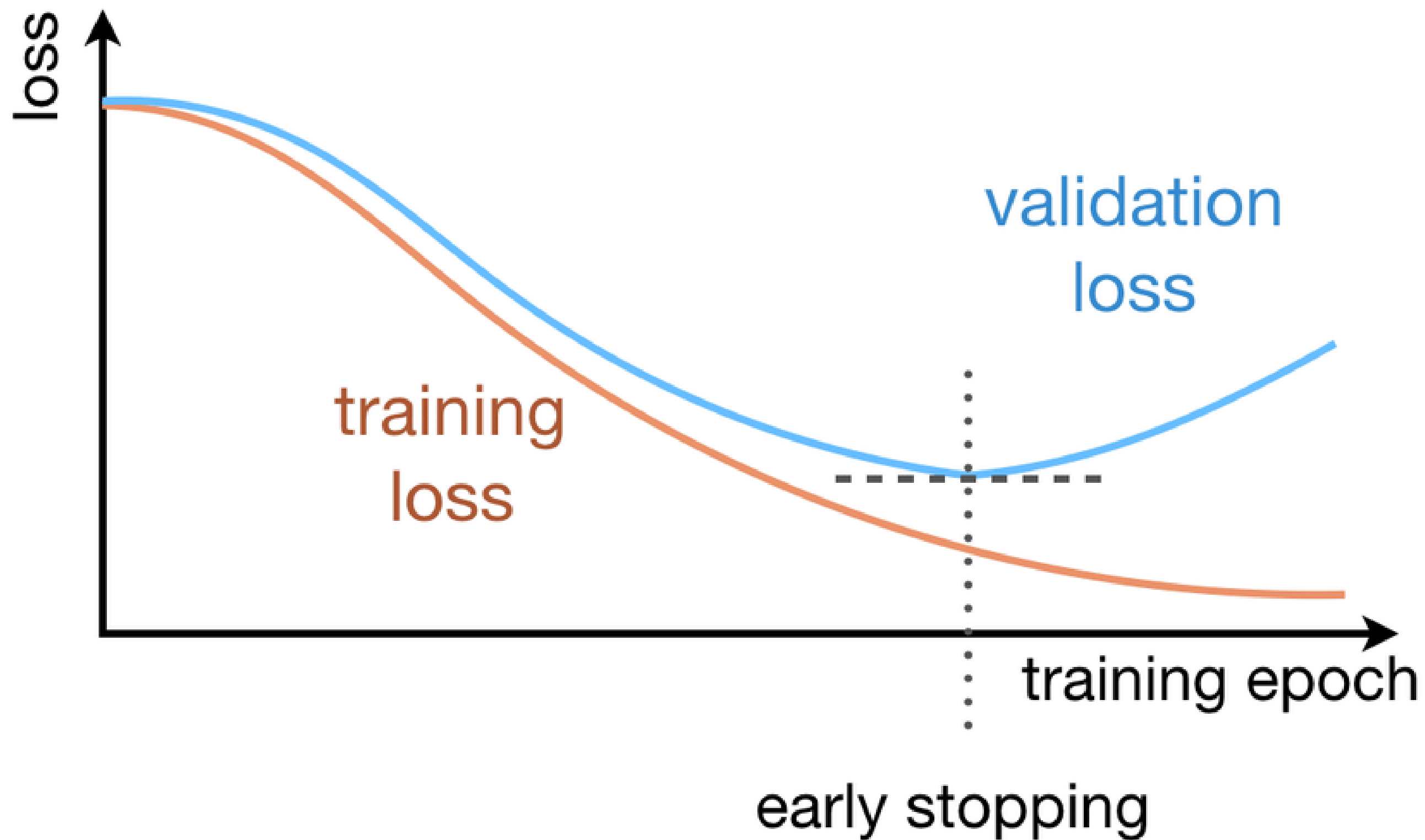
$$p^{[3]} = 0.0$$

- Durante el entrenamiento, “apagamos” aleatoriamente algunas neuronas.
- Esto evita que un modelo dependa demasiado de algunas neuronas.

Dropout



Early stopping



「muchas gracias.」